SE7EN Barbershop — Manual de Despliegue

Oct 27, 2026 · @HOLYEC PERU

Este manual deja en producción SE7EN Barbershop v12: base de datos PostgreSQL y backend Node.js en Railway, y la app Android (APK) instalada en el celular de cada barbero. Está escrito a partir del código de `Barber-v12.zip` (backend Express + `pg`, frontend React 18 + Vite empaquetado con Capacitor 8).

Arquitectura

La app del celular habla con el backend por HTTPS; el backend es el único que toca la base de datos. Las ventas y egresos del barbero se guardan en IndexedDB cuando no hay señal y se sincronizan solos al volver la conexión.

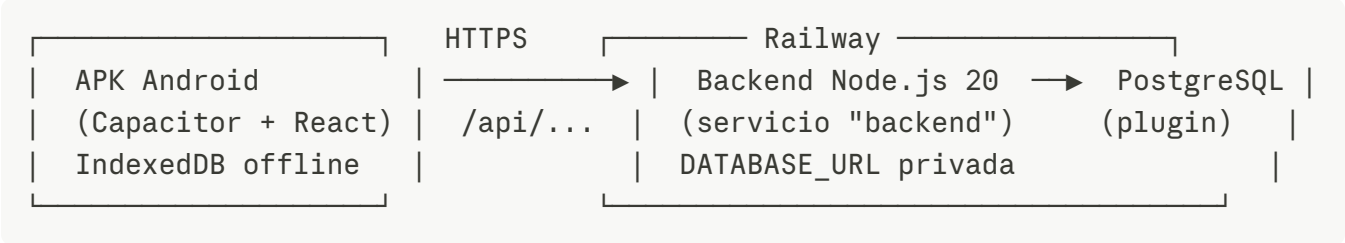


Tabla de contenidos

1. Requisitos previos
2. Base de datos PostgreSQL en Railway
3. Backend en Railway
4. Verificación y credenciales
5. Generación del APK
6. Instalación del APK en el celular
7. Actualizaciones, observaciones y solución de problemas

1. Requisitos previos

Para Railway solo necesitas cuentas web; para el APK necesitas una PC (Windows, Mac o Linux) con las herramientas de la tabla.

Herramienta	Versión	Para qué	Enlace
Cuenta GitHub	—	Solo para la opción A de la sección 3 (despliegue automático desde un repositorio)	github.com
Cuenta Railway	Plan Hobby o superior	Base de datos + backend; se entra con Google (Gmail) o GitHub	railway.app
Git	Cualquiera reciente	Subir el código a GitHub	git-scm.com
Node.js	22 LTS	Compilar el frontend; Capacitor 8 no funciona con Node 20 o anterior	nodejs.org
JDK	21	Gradle / compilación Android	adoptium.net
Android Studio	Versión estable actual	SDK de Android (API 36) y firma del APK	developer.android.com/studio
Celular Android	7.0 o superior (minSdk 24)	Instalar la app	—

> Android Studio trae su propio JDK 21 ("jbr"). Si compilas desde terminal sin Android Studio abierto, define `JAVA_HOME` apuntando a ese JDK o a uno 21 instalado aparte.

Subir el código a GitHub (solo opción A)

Railway necesita el código en un repositorio. Desde la carpeta descomprimida `Barber-v12`:

```
cd Barber-v12
git init
git add .
git commit -m "SE7EN Barbershop v12"
git branch -M main
git remote add origin https://github.com/TU-USUARIO/se7en-barbershop.git
git push -u origin main
```

> **Recomendado, no obligatorio.** El zip v12 no trae `.gitignore` en la raíz. El primer push sube limpio, pero en las actualizaciones la carpeta ya tendrá `node_modules/`, `dist/`, `android/app/build/` y el keystore `se7en.jks`, que no deben subirse. Crea el `.gitignore` con este contenido y deja el repositorio como **privado** (`seed.js` contiene la contraseña del admin):

```
node_modules/
.env
dist/
*.keystore
*.jks
frontend/android/app/build/
frontend/android/.gradle/
```

2. Base de datos PostgreSQL en Railway

La base se crea primero y vacía; las tablas las crea el backend con `migrate.js` en su primer arranque.

Paso 1 — Crear el proyecto

1. Entra a railway.app e inicia sesión con tu cuenta de **Google (Gmail)** o con **GitHub**. Cualquiera sirve; GitHub solo hace falta si usarás la opción A de la sección 3.
2. Pulsa **New Project** → **Empty Project**.
3. *(Opcional)* Renombra el proyecto. Railway le pone un nombre aleatorio (por ejemplo `charming-wonder`) y nada del código depende de él:
   a. Pulsa **Settings** arriba a la derecha, o haz clic en el nombre del proyecto arriba a la izquierda y elige **Settings**.
   b. En la pestaña **General**, en **Project Name**, escribe `se7en-barbershop`.

c. Pulsa **Update** (o **Save**). Si la interfaz cambió, busca el ícono de engranaje del proyecto.

Paso 2 — Agregar PostgreSQL

1. Dentro del proyecto pulsa **+ Create** (o **New**) → **Database** → **Add PostgreSQL**.
2. Espera a que el servicio `Postgres` quede en verde (**Active**).
3. Abre el servicio → pestaña **Variables**. Railway genera solas `DATABASE_URL`, `PGHOST`, `PGUSER`, `PGPASSWORD`, `PGDATABASE` y `PGPORT`. No las edites.

Variable	Uso
`DATABASE_URL`	Conexión por la red privada de Railway; es la que usará el backend
`DATABASE_PUBLIC_URL`	Conexión desde tu PC (pgAdmin, DBeaver, `psql`) para revisar datos o hacer backups

Paso 3 — Zona horaria (opcional)

El backend ya fuerza `America/Lima` en cada conexión (`SET TIME ZONE` en `pool.js`), así que no ha horace falta tocar la base. Si quieres que también las consultas manuales usen de Lima, en la pestaña **Data → Query** del servicio Postgres ejecuta:

```
ALTER DATABASE railway SET timezone TO 'America/Lima';
```

Paso 4 — Backups

En el servicio Postgres → **Backups**, activa backups programados (diarios). Para un respaldo manual desde tu PC:

```
pg_dump "<DATABASE_PUBLIC_URL>" -Fc -f se7en_$(date +%F).dump
```

3. Backend en Railway

El backend se despliega desde la carpeta `backend/` del repositorio; `backend/railway.json` ya define el build, el arranque (migrate → seed → server) y el health check en `/health`.

Paso 1 — Crear el servicio del backend

Elige una de las dos opciones. La A redespliega sola con cada `git push`; la B no necesita GitHub y sube el código desde tu PC.

Opción A — Desde GitHub (recomendada)

Sigues entrando a Railway con tu Gmail; GitHub solo queda conectado como fuente del código.

1. Crea una cuenta gratuita en github.com (puedes registrarte con el mismo Gmail) y sube el código como indica la sección 1.
2. En el proyecto de Railway pulsa **+ Create** → **GitHub Repo**. La primera vez aparecerá **Configure GitHub App**: autorízalo y da acceso al repositorio `se7en-barbershop`.
3. Elige el repositorio. Abre el servicio nuevo → **Settings** → **Source** → **Root Directory** → escribe `/backend`.
4. En **Settings** → **Config-as-code** → **Railway Config File** escribe `/backend/railway.json`. Railway no busca este archivo dentro del Root Directory por sí solo, y sin él usaría el `Procfile` (que solo ejecuta `node src/server.js` y nunca crearía las tablas).
5. Renombra el servicio como `backend`.

Opción B — Sin GitHub, con la CLI de Railway

1. En el proyecto de Railway pulsa **+ Create** → **Empty Service** y renómbralo como `backend`.
2. En tu PC instala la CLI e inicia sesión (abre el navegador; entras con tu Gmail):

```
npm install -g @railway/cli
railway login
```

3. Vincula la carpeta del backend con el proyecto y el servicio `backend`:

```
cd Barber-v12/backend
railway link
```

4. No configures **Root Directory**: ya estás dentro de `backend`, y Railway encuentra `railway.json` en la raíz de lo que subes.
5. Continúa con el Paso 2 (variables). El despliegue se lanza en el Paso 3 con `railway up`.

> En la opción B cada actualización del backend se sube a mano con `railway up` desde `backend/`.

Paso 2 — Variables de entorno

En el servicio `backend` → **Variables** → **Raw Editor**, pega:

```
DATABASE_URL=${{Postgres.DATABASE_URL}}
NODE_ENV=production
JWT_SECRET=PEGA_AQUI_EL_SECRETO_GENERADO
CORS_ORIGIN=*
LOG_LEVEL=info
TZ=America/Lima
```

Variable	Valor	Por qué
`DATABASE_URL`	`${{Postgres.DATABASE_URL}}`	Referencia a la base del paso 2; si tu servicio de BD tiene otro nombre, usa ese nombre
`NODE_ENV`	`production`	Activa SSL en la conexión a PostgreSQL (`pool.js`)
`JWT_SECRET`	128 caracteres hex	Firma los tokens de login del admin (8 h)
`CORS_ORIGIN`	`*`	La app Android llama desde el origen `https://localhost` ; ver la nota de abajo
`LOG_LEVEL`	`info`	Nivel de logs de winston
`TZ`	`America/Lima`	Fechas de ventas y egresos en hora de Lima

Genera el `JWT_SECRET` en tu PC y pégalo:

```
node -e "console.log(require('crypto').randomBytes(64).toString('hex'))"
```

> **No defines `PORT`.** Railway inyecta su propio `PORT` y `server.js` ya lo lee (`process.env.PORT || 3001`). Si fijas `PORT=3001` tendrás que configurar el puerto destino del dominio a mano.

> **CORS:** si en vez de `*` quieres restringir orígenes, el APK de Capacitor siempre llama desde `https://localhost`. El código actual acepta un solo origen en `CORS_ORIGIN`, así que restringirlo a la web dejaría fuera al APK. Deja `*` salvo que cambies `server.js` para aceptar una lista.

Paso 3 — Primer despliegue (crea tablas y datos iniciales)

1. Pulsa **Deploy** (o espera el despliegue automático tras guardar variables. En la opción B ejecuta railway up desde la carpeta backend).
2. En **Deployments → View Logs** deben aparecer, en orden:

```
🔄  Running migrations…
✅  Migrations completed successfully.
🌱  Seeding database…
✅  Seed completed successfully.
   Admin: hanz@seven.com / se7en2026
✅  DB connected (Lima time): ...
🚀  SE7EN API corriendo en puerto 8080 [production]
```

Paso 4 — Quitar el seed del arranque (importante)

Tras el primer despliegue correcto, quita `seed.js` del comando de arranque. El seed usa `ON CONFLICT DO NOTHING`, pero las tablas `servicios` y `productos` no tienen restricción única y en `usuarios` el `nro_documento` tampoco es único: cada redeploy o reinicio **duplica los 6 barberos, los 17 servicios y los 8 productos**.

Edita `backend/railway.json` y deja:

```
"startCommand": "node src/db/migrate.js && node src/server.js"
```

Luego redespliega: en la opción A, `git commit` y `git push` (Railway redespliega solo); en la opción B, `railway up` desde `backend/`. `migrate.js` usa `CREATE TABLE IF NOT EXISTS`, así que puede quedarse en cada arranque sin riesgo.

Paso 5 — Generar el dominio público

1. Servicio `backend` → **Settings** → **Networking** → **Generate Domain**.
2. Railway entrega una URL tipo `https://backend-production-xxxx.up.railway.app`.
3. Guárdala: es el `VITE_API_URL` que llevará el APK (sección 5).

4. Verificación y credenciales

El backend está listo cuando `/health` responde `ok` y el login del admin devuelve un token.

Health check

Abre en el navegador:

```
https://backend-production-xxxx.up.railway.app/health
```

Respuesta esperada:

```
{"status":"ok","timestamp":"2026-10-27T15:00:00.000Z","env":"production"}
```

Prueba de API

```
# Lista de barberos (pública) — deben salir 6
curl https://backend-production-xxxx.up.railway.app/api/barberos

# Login admin — debe devolver un token
curl -X POST https://backend-production-xxxx.up.railway.app/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"hanz@seven.com","password":"se7en2026"}'
```

> `/health` responde `ok` aunque la base esté caída: `server.js` arranca igual si `testConnection()` falla. Usa siempre `/api/barberos` para confirmar que la base responde.

Credenciales por defecto

Campo	Valor
Email	`hanz@seven.com`
Contraseña	`se7en2026`
Nombre	Hanz Ccorahua Aquiño

Barberos iniciales: CARLOS, PAOLO, OMER, JJ, WEIMBER, EDUARDO.

> ⚠️ Cambia la contraseña del admin después del primer login. La contraseña por defecto está escrita en `seed.js` y en este manual.

5. Generación del APK

El APK se genera con Capacitor: Vite compila el frontend a `dist/`, Capacitor lo copia al proyecto Android de `frontend/android/` y Gradle lo empaqueta. La URL del backend queda grabada dentro del APK al compilar, así que **el backend de Railway debe existir antes de este paso**.

Dato de la app	Valor
App ID	`com.se7en.barbershop`
Nombre	SE7EN Barbershop
Android mínimo	7.0 (minSdk 24)
Target / compile SDK	36
Versión	1.0 (versionCode 1)

Paso 1 — Instalar dependencias

```
cd Barber-v12/frontend
npm install
```

Paso 2 — Apuntar el APK al backend

El `frontend/.env` del zip apunta a un backend antiguo (`web-production-a617`) y trae un espacio y fin de línea de Windows al final. Reemplázalo completo con la URL del paso 5 de la sección 3, **sin barra final y con `https://`**:

```
# Windows (PowerShell)
Set-Content -Path .env -Value "VITE_API_URL=https://backend-production-
xxxx.up.railway.app" -NoNewline

# Mac / Linux
printf "VITE_API_URL=https://backend-production-xxxx.up.railway.app" > .env
```

> El APK bloquea HTTP sin cifrar (`usesCleartextTraffic="false"` en `AndroidManifest.xml`). Una URL `http://` o una IP local no funcionará.

Paso 3 — Compilar el frontend y sincronizar con Android

```
npm run cap:sync
```

Este script ejecuta `vite build` y luego `capacitor sync android`, que reemplaza la copia vieja que trae el zip en `android/app/src/main/assets/public/`. Salida esperada al final:

```
✔ Copying web assets from dist to android/app/src/main/assets/public
✔ Updating Android plugins
[info] Sync finished
```

Paso 4 — Indicar dónde está el SDK de Android

Abre Android Studio una vez para que instale el SDK 36. Luego, si vas a compilar desde terminal, crea `frontend/android/local.properties` con la ruta del SDK:

```
# Windows (ojo con las barras dobles)
sdk.dir=C\:\\Users\\TU_USUARIO\\AppData\\Local\\Android\\Sdk

# Mac
sdk.dir=/Users/TU_USUARIO/Library/Android/sdk
```

(Si abres el proyecto con `npm run cap:open`, Android Studio crea este archivo solo.)

Paso 5A — APK de prueba (debug)

Sirve para probar en tu propio celular; no lo distribuyas.

```
cd android

# Windows
.\gradlew.bat assembleDebug

# Mac / Linux
./gradlew assembleDebug
```

Resultado: `frontend/android/app/build/outputs/apk/debug/app-debug.apk`

> En Windows no uses `npm run android:build-debug`: el script llama a `./gradlew`, que solo funciona en Mac/Linux o Git Bash.

Desde Android Studio: **Build → Build App Bundle(s) / APK(s) → Build APK(s)**.

Paso 5B — APK de producción firmado (para los barberos)

1. Crear el keystore (una sola vez):

```
keytool -genkeypair -v -keystore se7en.jks -alias se7en_key \
  -keyalg RSA -keysize 2048 -validity 10000
```

Te pedirá una contraseña y datos de la empresa. Guarda `se7en.jks` y sus contraseñas fuera del repositorio y con copia de respaldo: **sin este archivo no podrás publicar actualizaciones** que se instalen encima de la app existente.

2. Compilar firmado:

```
cd android

# Windows (PowerShell, una sola línea)
.\gradlew.bat assembleRelease -
Pandroid.injected.signing.store.file=C:\ruta\se7en.jks -
Pandroid.injected.signing.store.password=TU_PASSWORD -
Pandroid.injected.signing.key.alias=se7en_key -
Pandroid.injected.signing.key.password=TU_KEY_PASSWORD

# Mac / Linux
./gradlew assembleRelease \
  -Pandroid.injected.signing.store.file=/ruta/se7en.jks \
  -Pandroid.injected.signing.store.password=TU_PASSWORD \
  -Pandroid.injected.signing.key.alias=se7en_key \
  -Pandroid.injected.signing.key.password=TU_KEY_PASSWORD
```

Resultado: `frontend/android/app/build/outputs/apk/release/app-release.apk`

Alternativa en Android Studio: Build → Generate Signed App Bundle / APK → APK → elige `se7en.jks` y el alias `se7en_key` → variante **release** → **Create**.

3. Comprobar la firma (opcional):

```
# apksigner viene en Android/Sdk/build-tools/<versión>/
apksigner verify --print-certs app-release.apk
```

6. Instalación del APK en el celular

Usa `app-release.apk` para los barberos; `app-debug.apk` solo para pruebas.

Opción A — Copiar el archivo (recomendada)

1. Envía `app-release.apk` al celular por WhatsApp, Google Drive, correo o cable USB.
2. Abre el archivo desde la notificación o desde **Archivos → Descargas**.
3. Android pedirá permiso para instalar apps de esa fuente: **Configuración → Permitir desde esta fuente** (en Android 7 es **Ajustes → Seguridad → Orígenes desconocidos**).
4. Pulsa **Instalar**. Si Google Play Protect muestra un aviso de "app desconocida", elige **Más detalles → Instalar de todas formas**.
5. Abre **SE7EN Barbershop** desde el cajón de aplicaciones.

Opción B — Por cable con ADB

1. En el celular activa **Opciones de desarrollador** (toca 7 veces **Número de compilación** en **Acerca del teléfono**) y luego **Depuración USB**.
2. Conecta el cable y acepta la huella de la PC en el celular.
3. Ejecuta:

```
adb devices
adb install -r frontend/android/app/build/outputs/apk/release/app-release.apk
```

Primera prueba en el celular

- [ ] La pantalla de inicio carga y muestra los 6 barberos (confirma conexión con Railway).
- [ ] Un barbero registra una venta de prueba con datos activos.
- [ ] Con el celular en modo avión, registra otra venta: debe quedar pendiente en la barra offline.
- [ ] Al quitar el modo avión, la venta pendiente se sincroniza sola.
- [ ] El admin entra con `hanz@seven.com` y ve ambas ventas en reportes.

> El primer arranque necesita internet para descargar barberos y catálogo; el modo offline funciona a partir de ahí y solo para el flujo del barbero. El panel de admin siempre requiere conexión.

7. Actualizaciones, observaciones y solución de problemas

Publicar una actualización

Backend: en la opción A haz `git push` a `main` y Railway redespliega solo; en la opción B ejecuta `railway up` desde `backend/`. En ambos casos `migrate.js` aplica tablas nuevas sin borrar datos.

APK:

1. En `frontend/android/app/build.gradle` sube `versionCode` (1 → 2 → 3…) y ajusta `versionName` ("1.1").
2. Repite `npm run cap:sync` y el **Paso 5B** con el **mismo** `se7en.jks`.
3. Instala el nuevo APK encima del anterior; los datos offline pendientes se conservan.

> Si Android dice "La app no está instalada" al actualizar, el APK se firmó con otra clave o con un `versionCode` menor. Desinstalar y reinstalar funciona, pero **borra las ventas offline no sincronizadas** de ese celular.

Observaciones de la revisión del código

#	Hallazgo	Archivo	Impacto	Acción en este manual
1	El seed corre en cada arranque y no hay restricciones únicas en barberos, servicios ni productos	`railway.json`, `seed.js`, `migrate.js`	Datos duplicados en cada redeploy	Sección 3, paso 4
2	Railway no lee `railway.json` dentro del Root Directory sin configurarlo; el `Procfile` no ejecuta migrate	`backend/railway.json`, `Procfile`	Tablas sin crear	Sección 3, paso 1
3	`.env` del frontend apunta a `web-production-a617` y trae espacio + CRLF	`frontend/.env`	APK conectado al backend equivocado	Sección 5, paso 2
4	`/health` no comprueba la base y el servidor arranca aunque la conexión falle	`server.js`	Health check en verde con la BD caída	Verificar con `/api/barberos`
5	`CORS_ORIGIN` acepta un solo origen; el APK usa `https://localhost`	`server.js`	Restringir CORS rompe el APK	Mantener `*`

#	Hallazgo	Archivo	Impacto	Acción en este manual
6	Contraseña del admin fija en el código	`seed.js`	Credencial conocida	Cambiarla tras el primer login
7	Falta la carpeta `public/icons/` que referencia el manifest PWA	`vite.config.js`	Solo afecta la versión web; el APK usa sus propios íconos `mipmap`	Agregar `icon-192.png` y `icon-512.png` si se publica la web
8	Capacitor 8 exige Node 22 y JDK 21; el README pide Node 20	`package.json`	`cap sync` falla con Node 20	Sección 1
9	No hay `.gitignore` en la raíz y `BUILD_APK.md` menciona `Barber-v11`	raíz del zip	Riesgo de subir `.env` y keystore	Sección 1

Solución de problemas

Síntoma	Causa probable	Solución
Deploy falla en el health check	El servidor no levantó o escucha otro puerto	Revisa logs; borra la variable `PORT` si la definiste
Logs: `DB connection failed`	`DATABASE_URL` mal referenciada	Usa `${{Postgres.DATABASE_URL}}` con el nombre exacto del servicio de BD
Logs: `relation "usuarios" does not exist`	Se usó el `Procfile` en vez de `railway.json`	Configura `/backend/railway.json` (sección 3, paso 1)
Barberos o servicios repetidos	Seed ejecutado varias veces	Aplica el paso 4 de la sección 3 y borra duplicados por SQL

Síntoma	Causa probable	Solución
La app abre pero no carga barberos	URL incorrecta en `.env` al compilar, o `http://`	Corrige `.env`, repite `cap:sync` y recompila
Error CORS en la app	`CORS_ORIGIN` distinto de `*`	Vuelve a `*`
`cap sync` falla con error de versión de Node	Node menor a 22	Instala Node 22 LTS
Gradle: `SDK location not found`	Falta `local.properties`	Sección 5, paso 4
Gradle: `Unsupported class file major version` o error de Java	JDK distinto de 21	Apunta `JAVA_HOME` al JDK 21
"App no instalada" al actualizar	Otra firma o `versionCode` menor	Usa el mismo keystore y sube `versionCode`

SE7EN Barbershop Club · Manual de despliegue v12 · 2026